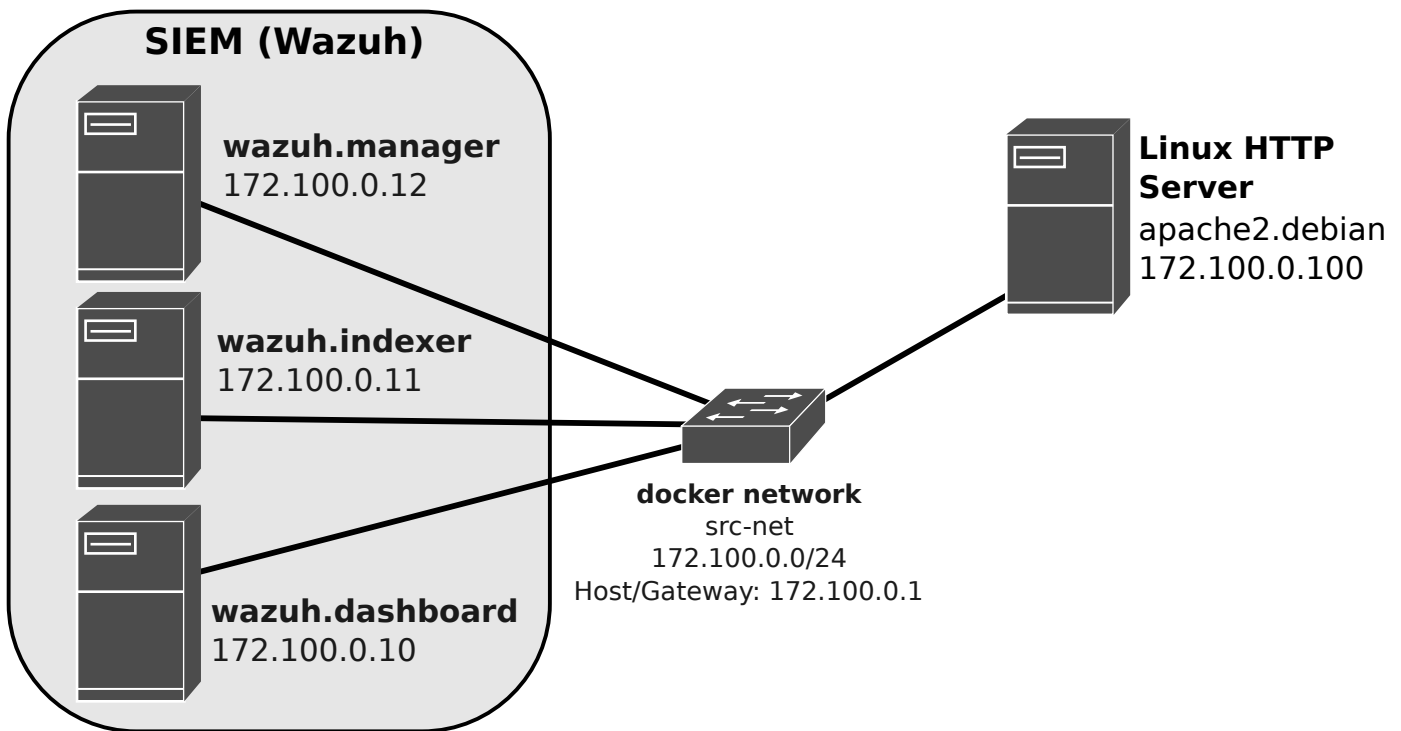




SEGURANÇA EM REDES DE COMUNICAÇÕES

NETWORK MONITORING & SIEM (WAZUH)



Download the provided docker compose and respective configuration files.

Syslog

1. Start the Linux HTTP Server docker (apache2):

```
docker compose up -d apache2
```

Verify that docker names apache2.debian is running, its running statistics:

```
docker ps
```

```
docker statistics
```

Note: To install additional packages on the apache2.debian docker, you may edit the file config_apache2/Dockerfile, stop the docker and restart it with the --build flag:

```
docker compose down apache2
```

```
docker compose up -d apache2 --build
```

2. Access the bash interface of the apache2.debian docker:

```
docker exec -it apache2.debian /bin/bash
```

Analyze the access log file from apache2:

```
watch cat /var/log/apache2/access.log
```

Using the host machine browser:

- Access the default apache2 web page using the browser on the host machine (URL: <http://172.100.0.100>).

- Force a full reload/download of the web page with Ctrl+F5.

- Request a specific existing object from the web page: (URL: <http://172.100.0.100/icons/openlogo-75.png>).

- Request a non-existing object from the web page: (URL: <http://172.100.0.100/icons/world.png>).

>> Analyze the default log entries format. More information [here](#).

>> Identify the individual objects accessed by the client (172.100.0.1).

>> Identify the response HTTP codes (200 and 404) for each request.

>> Identify the HTTP User-Agent reported by the client.

Exit the apache2.debian docker bash interface with:

```
exit
```

Rsyslog

3. By default (defined on the docker build file `config_apache2/Dockerfile`) the docker is already running rsyslogd service. The host file **`config_apache2/rsyslog.conf`** file, which is mapped to the internal docker file **`/etc/rsyslog.d/wazuh.conf`** has the following content:

```
# Load IMFile module to monitor log files
module(load="imfile")
# Watch Apache Access Log
input(type="imfile"
      File="/var/log/apache2/access.log"
      Tag="apache-access:"
      Severity="info"
      Facility="local7")
# Forward everything to an external server
*.* @172.100.0.1:514
```

These configurations tell rsyslog to monitor the Apache2 access log and perform a remote logging to IPv4 address 172.100.0.1 (using UDP port 514).

Identify the network id of the virtual bridge interface of the docker src-network with:

```
docker network ls
```

On the host machine, start a Wireshark capture on the bridge interface associated with the src-network, should be "br-" followed by the network id.

Using the host machine browser:

- Access the default apache2 web page using the browser on the host machine (URL: <http://172.100.0.100>).
- Force a full reload/download of the web page with Ctrl+F5.
- Request a specific existing object from the web page: (URL: <http://172.100.0.100/icons/openlogo-75.png>).
- Request a non-existing object from the web page: (URL: <http://172.100.0.100/icons/world.png>).

>> Analyze the captured rsyslog messages.

Note: without a UDP syslog input at the server, the server will respond to the rsyslog messages with a ICMP port unreachable message.

Stop the apache2.debian docker:

```
docker compose down apache2
```

Security Information and Event Management (SIEM)

The Wazuh SIEM is composed of three components:

- Wazuh Manager (wazuh.manger): central engine that receives data from agents and syslog sources, decodes the logs, and runs them against a ruleset to trigger security alerts. Its based on ELK Logstash.
- Wazuh Indexer (wazuh.indexer): high-performance, full-text search and analytical engine that stores and indexes the alerts generated by the manager. Its based on ELK Elasticsearch.
- Wazuh Dashboard (wazuh.dashboard): web interface used to visualize security events, monitor compliance, and manage Wazuh. Its based on ELK Kibana.

Important: If deploying the docker environment inside a VM, the **VM must have at least 4GB of RAM, 2 cores, and available disk space of approximately 13-15GB.**

You may free space, cleaning cache from apt installs:

```
sudo apt clean
```

As last resort delete all existing Docker images and containers [this will delete all previously used docker containers and images]:

```
docker system prune -af
```

You may need to increase the maximum number of **memory map areas** a process may have:

```
sudo sysctl -w vm.max_map_count=262144
# Permanent (survives reboots)
echo "vm.max_map_count=262144" | sudo tee -a /etc/sysctl.conf
sudo sysctl -p
```

4. First it is required to generate all operational security certificates. Run:

```
docker compose -f generate-indexer-certs.yml run --rm generator
```

Edit the apache2.debian docker rsyslog configuration file (**config_apache2/rsyslog.conf**) to send the rsyslog messages to the wazuh.manager docker (172.100.0.12):

```
...
# Forward everything to an external server
*. * @172.100.0.12:514
```

Start all dockers with:

```
docker compose up
```

Verify that all dockers are running and its running statistics:

```
docker ps
docker statistics
```

Verify if the Wazuh manager services are up using:

```
docker exec wazuh.manager /var/ossec/bin/wazuh-control status
```

Note: in this deployment not all services will be running.

5. This deployment of Wazuh is configured to log all received information for initial debug. See file base Wazuh configuration file (**config_wazuh/wazuh_cluster/ossec.conf**) and look for

```
<alerts_log>yes</alerts_log>
<logall>yes</logall>
```

Without this configuration, Wazuh will only store events that are considered relevant, i.e., where the rule/event level is equal or above 3.

Access the default apache2 web page using the browser on the host machine (URL: <http://172.100.0.100>). And inspect the archives.log (*.json) files:

```
docker exec wazuh.manager tail -f /var/ossec/logs/archives/archives.log
docker exec wazuh.manager tail -f /var/ossec/logs/archives/archives.json
```

>> Are the rsyslog messages being received by Wazuh manager?

>> Have the archived events in Wazuh any additional information?

>> The archives.json file contains all fields properly decoded?

Note: archive.log/json files are only required for initial debug, and consumes storage space fast. After verifying that rsyslog messages are being logged, in the Wazuh configuration file (**config_wazuh/wazuh_cluster/ossec.conf**) change to:

```
<alerts_log>no</alerts_log>
<logall>no</logall>
```

6. Access Wazuh Dashboard using your browser using URL <https://172.100.0.10:5601>. Use the default login/password: kibanaserver.

Simulate the following attacks against the Apache2 server (in bash):

- Path Traversal (Wazuh rule 31101 – level 5)

```
curl -XGET "http://172.100.0.100/../../../../etc/passwd"
```
- XSS (Cross Site Scripting) attempt (Wazuh rule 31105 – level 6)

```
curl -XGET "http://172.100.0.100/search?q=<script>alert(1)</script>"
```
- SQL injection attempt (Wazuh rule 31164 – level 6)

```
curl -G "http://172.100.0.100/index.php" --data-urlencode "id=' OR '1'='1"
```
- Shellshock attack (Wazuh rule 31168 – level 15)

```
curl -H "User-Agent: () { : }; echo; /bin/l" http://172.100.0.100/
```
- Multiple web server 400 error codes from same source (Wazuh rule 31151 – level 10)

```
for i in {1..20}; do curl http://172.100.0.100/protected/index.php; done
```

Go to Menu → Explore → Discover, and search for events from rules 31101, 31105, 31164, 31168 and 31151 using DQL queries: "rule.id: 31101", "rule.id: 31101", etc...

>> Expand the events data and analyze the contents of each report, namely identify description of attack, target of the attack and source of the attack.

>> Analyze the MITRE section of the event. Identify the characteristics of the attack. Go to [MITRE ATT@Ck](#) and search for the attack id (*rule.mitre.id*).

>> Go to Menu → Server management → Rules. Search the default rules 31101, 31105, 31164, 31168 and 31151. Analyze the respective rules syntax and description

>> Go to Menu → Server management → Rules. Search the default rules web rules (0245-web_rules.xml). Identify more attacks detected by Wazuh by default.

>> Explore other default Wazuh rules.

SIEM Custom Rules

7. Make a less intensive attack to the Apache2 server:

```
for i in {1..5}; do curl http://172.100.0.100/protected/index.php; done
```

Go to Menu → Explore → Discover, and search for events from rule 31151 using DQL queries: "rule.id: 31151".

>> Does Wazuh generate an alert?

8. Improve multiple 400 error rule by creating a custom rule, overwriting default rule 31151, to detect at least four 400 errors in 10 seconds. Go to Menu → Server management → Rules → Custom rules, then click local_rules.xml to edit, and add:

```
<group name="web,accesslog,">
  <rule id="31151" level="11" frequency="4" timeframe="10" overwrite="yes">
    <if_matched_sid>31101</if_matched_sid>
    <same_source_ip />
    <description>Multiple web server error codes from same source ip (custom).</description>
    <mitre>
      <id>T1595.002</id>
    </mitre>
  </rule>
</group>
```

Save and reload rules.

Remake the less intensive attack to the Apache2 server:

```
for i in {1..5}; do curl http://172.100.0.100/protected/index.php; done
```

Verify if events for rule 31151 were now created (DQL - rule.id: 31151).

>> What can you conclude about the new rule effectiveness?

9. Many websites have hidden links to trigger files (not visible to human in browser, but visible to bots/scripts in html).

Create a custom rule to detect such accesses. Assume for simplicity that the trigger file is openlogo-75.png.

Go to Menu → Server management → Rules → Custom rules, then click local_rules.xml to edit, and add:

```
<group name="web,apache,custom">
  <rule id="100101" level="12">
    <if_sid>31108</if_sid>
    <id>^200$</id>
    <url>openlogo-75.png</url>
    <description>Apache: Trigger hidden image requested ($(url))</description>
    <group>web_trigger_access</group>
  </rule>
</group>
```

Save and reload rules.

Test the rule just accessing to the default web page of Apache2 server (URL: <http://172.100.0.100>).

Go to Menu → Explore → Discover, and search for events from rule 100101 using DQL queries: "rule.id: 100101".

>> What can you conclude about the usage of such rule in a real environment?

>> What counter measures should be deployed?

Note: default rule 31108 is an alert level zero rule used to identify non-error HTTP requests. This custom rule will be evaluated after to increase the alert level to 12 (High Severity).

10. To identify DDoS sources many websites have visible trigger files, that when repeatedly requested, an high severity alert must be created. Again, assume for simplicity that the trigger file is openlogo-75.png. Assume that an alert level 13 should be created when four requests are made in less than 30 seconds from the same IP address.

Go to Menu → Server management → Rules → Custom rules, then click local_rules.xml to edit, and add new rule 100102:

```
<group name="web,apache,custom">
  <rule id="100101" level="12">
    <if_sid>31108</if_sid>
    <id>^200$</id>
    <url>openlogo-75.png</url>
    <description>Apache: Trigger hidden image requested ($(url))</description>
    <group>web_trigger_access</group>
  </rule>
  <rule id="100102" level="13" frequency="4" timeframe="30">
    <if_matched_sid>100101</if_matched_sid>
    <same_srcip />
    <description>Possible DDoS: Trigger hidden image requested ($(url)).</description>
    <group>web_trigger_access_ddos</group>
  </rule>
</group>
```

Save and reload rules.

Make a simulated DDoS attack to the Apache2 server:

```
for i in {1..5}; do curl -s http://172.100.0.100/icons/openlogo-75.png; done
```

Go to Menu → Explore → Discover, and search for events from rule 100101 using DQL queries: "rule.id: 100102".

>> What can you conclude about the usage of such rule in a real environment?

>> What counter measures should be deployed?

SIEM Custom decoder

11. Assume that exists an external tool that analyses end-devices behaviors (UEBA) and reports any anomaly to the Wazuh manager via rsyslog. To send a custom rsyslog message to the Wazuh manager, run from host:

```
logger -n 172.100.0.12 -P 514 "Alarm UEBA 10.1.1.1"
```

To parse such custom messages a custom decoder must be added. Go to Menu → Server management → Decoders → Custom decoders, then click local_decoder.xml to edit, and add new decoder for UEBA rsyslog:

```
<decoder name="ueba_alarm">
  <prematch>Alarm UEBA</prematch>
</decoder>

<decoder name="ueba_alarm_fields">
  <parent>ueba_alarm</parent>
  <regex offset="after_parent">(\d+.\d+.\d+.\d+)</regex>
  <order>srcip</order>
</decoder>
```

Save and reload the decoders.

After, it is required to create a new rule to make a level 7 alert every time the external tool identifies an anomalous device. Go to Menu → Server management → Rules → Custom rules, then click local_rules.xml to edit, and add new rule 100201:

```
<group name="syslog,ueba,">
  <rule id="100201" level="7">
    <decoded_as>ueba_alarm</decoded_as>
    <description>UEBA Alarm triggered from $(srcip)</description>
    <group>ueba_security_event</group>
  </rule>
</group>
```

Save and reload rules.

Test from the host machine:

```
logger -n 172.100.0.12 -P 514 "Alarm UEBA 10.1.1.1"
```

```
logger -n 172.100.0.12 -P 514 "Alarm UEBA 200.1.1.1"
```

>> What can you conclude about the usage of such rule in a real environment?